

SysY Runtime Library (2022 Edition)

The SysY runtime library provides a set of I/O functions, timing functions, and other facilities used to express input/output, timing, and similar requirements in SysY programs. These library functions can be used inside SysY functions without being declared in the SysY program itself. It should be noted that some SysY library functions accept parameter types beyond the data types supported by SysY, such as strings. A SysY compiler must be able to handle such cases and correctly pass these arguments from a SysY program to the SysY runtime library.

1. Files Related to the Runtime Library

The contest organizers provide the following SysY runtime library files to contestants:

- `libsysy.a` and `libsysy.so` are the static and dynamic versions of the SysY runtime library, respectively, for the contest target platform. For fairness, final evaluation will uniformly use static linking.
- `sylib.h` contains declarations of the functions and other items involved in the SysY runtime library.

Note: SysY source programs do not include `sylib.h`; instead, the SysY compiler analyzes and handles calls to these functions in SysY programs.

2. I/O Functions

The SysY runtime library provides a series of I/O functions that support input and output of integers, floating-point numbers, characters, and sequences of integers or floating-point numbers. To make it easier to control output format in SysY programs, I/O functions such as `putf` accept parameters beyond the data types supported by the SysY language, such as format strings.

The SysY runtime library provides the following I/O functions. Their arguments may be integer values, floating-point values, variables, or array element access expressions.¹

1. `int getint()`

Reads an integer and returns its value.

Example: `int n; n = getint();`

2. `int getch()`

Reads a character and returns its ASCII code.

Example: `int n; n = getch();`

3. `float getfloat()`

Reads a floating-point number and returns its value.

Example: `float n; n = getfloat();`

¹Compared with the 2020 edition, SysY2022 adds the `float` basic type and supports multidimensional arrays whose element type is `float`.

4. `int getarray(int[])`

Reads a sequence of integers. The first integer indicates how many integers follow; this count is returned as the function result. The subsequent integers are returned through the array argument.

Note: `getarray` obtains the starting address of the array passed in. It does not check whether the caller-provided array has enough space to hold the input sequence.

Example: `int a[10][10]; int n; n = getarray(a[0]);`

5. `int getfarray(float[])`

Reads an integer followed by several floating-point numbers. The first integer indicates how many floating-point numbers follow; this count is returned as the function result. The subsequent floating-point numbers are returned through the array argument.

Note: `getfarray` obtains the starting address of the array passed in. It does not check whether the caller-provided array has enough space to hold the input sequence.

Example: `float a[10][10]; int n; n = getfarray(a[0]);`

6. `void putint(int)`

Outputs the value of an integer.

Example: `int n = 10; putint(n); putint(10); putint(n);`

Output: 101010

7. `void putch(int)`

Interprets the integer argument as an ASCII code and outputs the corresponding character.

Note: The valid range of the integer argument is 0 to 255. `putch()` does not check argument validity.

Example: `int n = 10; putch(n);`

Output: a newline character

8. `void putfloat(float)`

Outputs the value of a floating-point number.

Example: `float n = 10.0; putfloat(n); putfloat(10.0); putfloat(n);`

Output: 10.00000010.00000010.000000

9. `void putarray(int, int[])`

The first argument indicates how many integers are to be output (suppose this number is N). It should be followed by an array containing the N integers to output. `putarray` inserts spaces between integers when printing.

Note: `putarray` does not check argument validity.

Example: `int n = 2; int a[] = {2, 3}; putarray(n, a);`

Output: 2:2 3

10. `void putfarray(int, float[])`

The first argument indicates how many floating-point numbers are to be output (suppose this number is *N*). It should be followed by an array containing the *N* floating-point numbers to output. `putfarray` inserts spaces between floating-point numbers when printing.

Note: `putfarray` does not check argument validity.

Example: `int n = 2; float a[] = {2.0, 3.0}; putfarray(n, a);`

Output: 2:2.000000 3.000000

11. `void putf(<format string>, int, ...)`

The first argument is a format string that may contain only three format specifiers: `%d`, `%c`, and `%f`. The function outputs according to the format string. Ordinary characters are output as-is. When `%d`, `%c`, or `%f` is encountered, the function takes the corresponding argument starting from the second parameter and outputs it as an integer, character, or floating-point number, respectively.

Example:

```
int n = 2;
int a[] = {2, 3};
putf("%d: %d(%c), %d(%c)", n, a[0], a[0] + 48, a[1], a[1] + 48);
```

Output: 2:2(2), 3(3)

3. Timing Functions

The SysY runtime library provides a pair of functions, `starttime` and `stoptime`, for timing the execution of a segment of code in SysY. A SysY program may insert multiple pairs of `starttime/stoptime` calls to obtain the execution time of the code between each pair, and the cumulative execution time of all such timed segments is reported when the SysY program finishes execution. Note that `starttime` and `stoptime` do not support nested calls; that is, a call sequence such as `starttime() ... starttime() ... stoptime() ... stoptime()` is not supported.

The timing interfaces are as follows:

12. `void starttime()`

Starts the timer. This function should be used together with `stoptime()`.

13. `void stoptime()`

Stops the timer. This function should be used together with `starttime()`. When the program finishes, it outputs the time spent by each timer and the cumulative total for all timers.

Format: `Timer#number@start-line-stop-line: hour-minute-second-microsecond`

Example:

```
void foo(int n) {  
    starttime();  
    for (int i = 0; i < n; i++) system("sleep 1");  
    stoptime();  
}  
  
int main() {  
    starttime();  
    for (int i = 0; i < 3; i++) system("sleep 1");  
    stoptime();  
    foo(2);  
}
```

Output:

Timer#001@0010-0012: 0H-0M-3S-3860us

Timer#002@0005-0007: 0H-0M-2S-2660us

TOTAL: 0H-0M-5S-6520us